PDE-Agents: An LLM-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning

Sayan Adhikari, Gulshan Noorsumar, Øyvind Jensen
MatPro, Institute for Energy Technology
[email]

Submitted: April 16, 2026

Abstract

We present *PDE-Agents*, a multi-agent ecosystem that automates the full lifecycle of finite element method (FEM) simulations through natural-language interaction. The system combines three specialist large language model (LLM) agents—Simulation, Analytics, and Database—orchestrated via a LangGraph supervisor graph, with a locally-deployed open-source LLM stack (Qwen3-Coder-Next, Llama 4 Scout) running on dual NVIDIA RTX PRO 6000 Blackwell GPUs (≈196 GB combined VRAM). The architecture is model-agnostic; we report cross-model validation across two generations of open-source LLMs. A central novelty is the integration of a *GraphRAG* knowledge base (Neo4j + 768-dimensional vector embeddings) that enriches agents with curated material properties, known failure patterns, and prior run lineage. We report five empirical contributions: (i) a formal verification and validation (V&V) study confirming second-order spatial convergence ($\mathcal{O}(h^2)$) for all three benchmark cases on the heat equation solver; (ii) a four-way ablation study across 17 benchmark tasks comparing KG On, KG Off, KG Smart, and a Prompt Inject baseline, revealing that mandatory KG invocation paradoxically *reduces* success to 59% versus a 100% LLM-only baseline, but that our proposed *KG Smart* integration—combining HNSW-based warm-start injection with lazy conditional retrieval—recovers 94% success while retaining knowledge graph access; (iii) a novel-material experiment using three fictional materials whose properties exist only in the KG, with an error propagation analysis showing that fabricated properties cause output deviations up to 949 K, while KG Smart achieves perfect material property fidelity (1.00) versus 0.34 for the KG-free baseline; (iv) an adaptive KG decision framework (Algorithm 1) that selects the optimal retrieval mode with 100% accuracy across all 17 tasks, validated by cross-model experiments on both `qwen2.5-coder:32b` and `qwen3-coder-next`; and (v) production-scale agent quality metrics from 1,135 real simulation runs showing 97.4% overall success and a 57.6% first-try rate. All code, models, and evaluation artifacts are released openly. Our findings demonstrate that *integration pattern*, not knowledge content, determines whether GraphRAG augmentation helps or hinders LLM agents, and identify concrete design principles for autonomous FEM simulation assistants.

Keywords: Large language model agents; finite element method; knowledge graph; GraphRAG; multi-agent systems; scientific computing automation; verification and validation.

1 Introduction

Finite element method simulations are central to engineering analysis across structural mechanics, heat transfer, fluid dynamics, and electromagnetics. Yet the gap between a domain expert's intent—"what if I use aluminium instead of steel?"—and a running, verified simulation involves a sequence of error-prone manual steps: geometry creation, boundary condition specification, solver parameter selection, result interpretation, and iterative debugging.

The recent emergence of *reasoning-capable* LLMs [1–3] has opened the possibility of agents that can autonomously traverse this workflow. Prior work on "AI for scientific computing" has largely focused on surrogate modelling [4–6], operator learning [7], or dataset-specific fine-tuning [8]. Much less attention has been paid to *procedural automation*—using LLMs to *set up and manage* solvers rather than to replace them.

Meanwhile, the *retrieval-augmented generation* (RAG) paradigm [9] has demonstrated that grounding LLM outputs with authoritative external knowledge dramatically reduces hallucination. The graph-structured variant, GraphRAG [10], further enables multi-hop reasoning over relational knowledge—particularly relevant in engineering contexts where material properties, mesh requirements, and solver stability rules form a rich, interconnected knowledge base.

In this paper we make the following contributions:

1. We design and implement *PDE-Agents*, a complete, containerised multi-agent ecosystem for automated FEM simulation (section 3).

2. We describe a novel *GraphRAG integration pattern* that encodes material properties, known failure modes, and prior run lineage as a Neo4j knowledge graph with HNSW vector-indexed embeddings (section 4).

3. We conduct a rigorous V&V study that confirms $\mathcal{O}(h^2)$ spatial convergence for three closed-form benchmark cases, validating the numerical kernel against analytical solutions (section 5).

4. We run a four-way ablation study (KG On, KG Off, KG Smart, Prompt Inject) isolating the KG contribution, identify a paradoxical performance *inversion* with mandatory KG invocation, and propose a *KG Smart* integration that recovers 94% success via warm-start injection and lazy retrieval (section 6).

5. We introduce a *novel-material experiment* using three fictional materials absent from any LLM training corpus, demonstrating that KG Smart achieves 2.9× higher material property fidelity than the KG-free baseline, which fabricates physically incorrect properties (section 6.4).

6. We report production-scale agent quality metrics derived from 1,135 real simulation runs (section 7).

2 Related Work

LLM agents for scientific computing. ChemCrow [11] demonstrated that equipping an LLM with chemistry tools (RDKit, reaction planners) enables autonomous laboratory-scale reasoning. SciAgent [12] and similar systems extend this to broader scientific question answering. Our work targets *numerical simulation* rather than knowledge retrieval, requiring tool-call pipelines that produce and verify deterministic numerical outputs.

Autonomous FEM systems. FEniCS [13] and its successor DOLFINx/FEniCSx [14] provide Python-scriptable FEM solvers that are amenable to LLM-driven automation. Recent work on "simulation copilots" [15, 16] shows that LLMs can generate solver scripts from natural language with moderate success, but these approaches are single-turn and lack closed-loop debugging. PDE-Agents provides a multi-turn, self-correcting agent that can iteratively debug failed simulations.

Multi-agent orchestration. LangGraph [17] implements graph-structured agent workflows that support conditional routing and persistent state, enabling the supervisor pattern we adopt. AutoGen [18] and CrewAI [19] offer related frameworks; we choose LangGraph for its explicit state graph semantics and tight LangChain integration.

Retrieval-augmented generation for engineering. RAG [9] has been applied to engineering documentation retrieval [20], code generation [21], and more recently to simulation parameter suggestion [22]. GraphRAG [10] adds structured relational traversal. Corrective RAG (CRAG) [23] introduces adaptive retrieval decisions based on document relevance scoring, and AriGraph [24] demonstrates episodic knowledge-graph memory for LLM agents. Our *KG Smart* pattern draws on both: warm-start injection from similar past runs (episodic memory) with lazy conditional tool invocation (corrective retrieval). To our knowledge, ours is the first system to combine GraphRAG with a live, iterative FEM simulation loop.

Knowledge graphs for materials science. Materials-focused knowledge graphs (e.g. MatKG [25], OPTIMADE [26]) encode material properties in RDF/OWL formats. Our Neo4j KG is lighter-weight and purpose-built for run-time agent queries, prioritising latency over completeness.

3 System Architecture

3.1 Design Philosophy

PDE-Agents adopts four guiding principles:

1. **Open and local:** All models run locally via Ollama on dual NVIDIA RTX PRO 6000 Blackwell GPUs (≈196 GB VRAM, CUDA 13.1), eliminating API costs and data-privacy concerns.

2. **Verifiable:** Every agent reasoning step, tool call, and result is logged to a relational database, enabling post-hoc audit.

3. **Containerised:** All 11 services are orchestrated via Docker Compose, ensuring reproducible deployment.

4. **Separation of concerns:** The FEM solver, LLM stack, knowledge graph, and persistence layer are decoupled services communicating over a private bridge network.

3.2 Agent Stack

Each agent's LLM backend is configurable via environment variables, enabling drop-in model upgrades without code changes. table 1 lists the default models and the alternatives validated in the cross-model experiments of section 6.4.

Table 1: Agent model configuration. Current defaults reflect the model upgrade from the initial (v1) deployment. The ablation study was conducted on both model generations.

Agent	Env Variable	Default (current)	Previous (v1)
Orch.	`ORCHESTRATOR_MODEL`	`llama4:scout`	`llama3.3:70b`
Sim.	`SIM._AGENT_MODEL`	`qwen3-coder-next`	`qwen2.5-coder:32b`
Anal.	`ANAL._AGENT_MODEL`	`llama4:scout`	`llama3.3:70b`
DB	`DB_AGENT_MODEL`	`qwen3-coder:30b`	`qwen2.5-coder:14b`

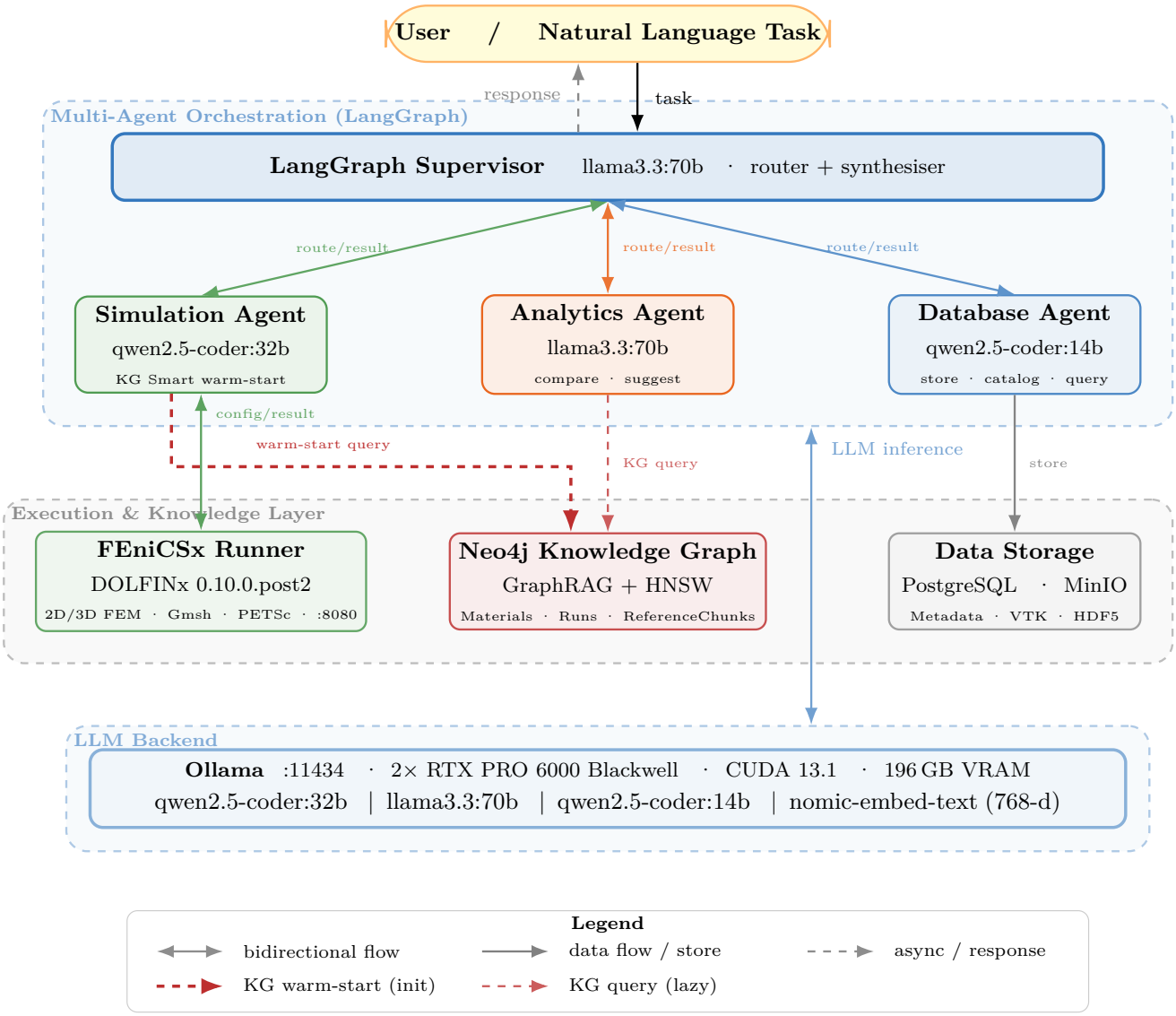

Figure 1: System architecture of PDE-Agents (four-tier layout). *Tier 1*: user natural-language interface. *Tier 2*: Multi-Agent Orchestration — a LangGraph Supervisor routes tasks to three specialist agents (Simulation, Analytics, Database), each backed by a locally-deployed LLM via Ollama. *Tier 3*: Execution & Knowledge — FEniCSx FEM runner (DOLFINx 0.10.0.post2), Neo4j GraphRAG Knowledge Graph (HNSW-indexed 768-d embeddings), and persistent storage (PostgreSQL + MinIO). *Tier 4*: shared Ollama LLM Backend (196 GB VRAM, CUDA 13.1). The thick dashed red arrow (Simulation Agent → Neo4j) represents the *KG Smart warm-start*: the top-3 similar past runs are retrieved via HNSW vector search and injected into the agent's prompt before the reasoning loop. The thin dashed red arrow (Analytics Agent → Neo4j) is the standard lazy KG query path. The blue bidirectional arrow at right carries LLM inference traffic between the orchestration layer and the shared Ollama backend.

Orchestrator. A LangGraph `StateGraph` supervisor receives the user's natural-language request and routes it to one of three specialist agents via structured JSON decisions. After each agent reports its result the supervisor either routes to another agent or synthesises a final response.

Simulation Agent. The core reasoning agent, executing a ReAct loop [3] with up to 15 reasoning steps and nine tools: `check_config_warnings`, `query_knowledge_graph`, `validate_config`, `run_simulation`, `modify_config`, `debug_simulation`, `list_recent_runs`, `get_run_status`, `run_parametric_sweep`.

Analytics Agent. Max 12 iterations; tools for statistical comparison across runs, sensitivity analysis, and LLM-generated textual summaries.

Database Agent. Max 10 iterations; answers history queries against PostgreSQL, cataloguing results and tracing run lineage.

3.3 Tool-Call Compatibility

Supporting multiple LLM families requires robust tool-call parsing, as models encode tool invocations differently. The Qwen 2.5-Coder family emits tool calls as JSON text in the `content` field rather than in the structured `tool_calls` field expected by LangChain's `ToolNode`. We implement a four-pass normalisation procedure in `BaseAgent._parse_content_tool_call`: (1) attempt `json.loads` on the full content; (2) extract from markdown fences; (3) extract the first `{...}` block; (4) repair truncated JSON via progressive bracket-closing and regex fallback.

A separate compatibility issue emerged with Qwen3-Coder-Next: the model occasionally *double-encodes* JSON tool arguments (e.g. `"{\"key\": \"val\"}"`). All tool functions now use a `_safe_json_parse` helper that detects and recursively unwraps such double-encoded strings, ensuring seamless operation across model families without model-specific workarounds. Llama 4 Scout, by contrast, uses the standard `tool_calls` field natively and requires no special parsing.

3.4 FEM Solver: DOLFINx Heat Equation

The FEniCSx runner container (DOLFINx 0.10.0.post2) exposes a FastAPI endpoint that accepts a `HeatConfig` JSON and returns a result including run metadata. The solver uses P1 (linear Lagrange) elements, with Backward Euler ($\theta = 1$, unconditionally stable) as the default time integration scheme, supporting Dirichlet, Neumann, and Robin boundary conditions. Geometry is built with Gmsh [27] for 2D and 3D domains from a library of nine parameterised types: `rectangle`, `l_shape`, `circle`, `annulus`, `hollow_rectangle`, `t_shape`, `stepped_notch`, `box`, and `cylinder`; Gmsh geometries use named physical groups for boundary identification, enabling arbitrary complex domain topologies.

4 Knowledge Graph Architecture

The Neo4j knowledge graph (Neo4j 5 Community Edition) serves three functions:

1. **Material property lookup.** A curated library of materials (metals, ceramics, polymers) with thermal conductivity k, density ρ, and specific heat c_p ranges, sourced from engineering handbooks and validated against ASM International data. Approximate values: steel $k \approx 50\,\mathrm{W/mK}$, copper $k \approx 385\,\mathrm{W/mK}$, aluminium $k \approx 200\,\mathrm{W/mK}$.

2. **Failure pattern catalogue.** `KnownIssue` nodes encode failure modes observed during system operation. A pure-Python rule engine (`rules.py`) fires nine pre-run checks analytically before each simulation:

Rule code	Trigger
`INCONSISTENT_IC`	$\lvert u_{\mathrm{init}} - T_{\mathrm{BC,min}} \rvert > 100\,\mathrm{K}$
`EXPLICIT_CFL_VIOLATION`	$\theta < 0.5$ and $\Delta t > h^2/(2\alpha)$
`NEAR_EXPLICIT_SCHEME`	$0 \le \theta < 0.5$
`COARSE_MESH_2D`	$n_x < 10$ or $n_y < 10$
`COARSE_MESH_3D`	any direction < 8
`SHORT_SIMULATION`	$t_{\mathrm{end}} < 5\,\Delta t$
`INVALID_MATERIAL_PROPS`	k, ρ or $c_p \le 0$
`LARGE_DT_RELATIVE_DIFFUSION`	$\Delta t > 10\,h^2/\alpha$
`NO_BOUNDARY_CONDITIONS`	`bcs` list empty

3. **Run lineage and semantic retrieval.** Each completed `Run` node is embedded with `nomic-embed-text` (768 dimensions) and indexed in an HNSW vector index. Up to k=5 `SIMILAR_TO` edges are created to the nearest neighbours with cosine similarity ≥ 0.85, precomputing the neighbourhood graph for fast agent traversal. The Simulation Agent can query "find similar past runs" to retrieve configurations with known outcomes, enabling few-shot in-context learning from the system's own history.

Document intelligence pipeline. A Celery-backed ingestion pipeline processes PDF papers, standards documents, and HTML references using Docling [28] for structured extraction, splitting them into `ReferenceChunk` nodes with context-aware 256-token windows (overlap 32 tokens) and HNSW-indexed embeddings. Each chunk is automatically cross-referenced to semantically similar `Run` nodes via `CROSS_REFS` edges (cosine ≥ 0.78), bridging literature knowledge to simulation history. As a representative scale point: indexing the 50-page FEniCSx tutorial produced 298 chunks and 1,073 cross-references.

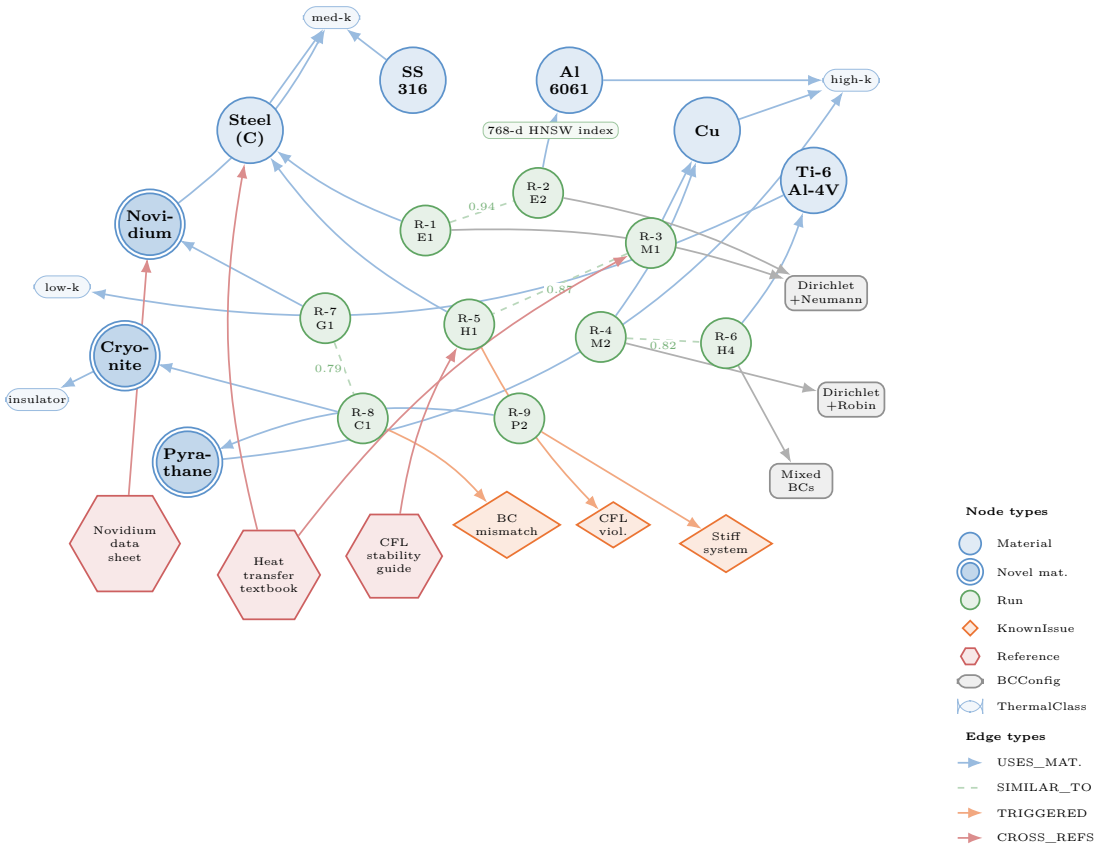

Figure 2: Knowledge graph visualisation (Neo4j-style). **Blue circles**: material nodes (double border = novel/-fictional materials known only to the KG). **Green circles**: simulation run nodes linked to their materials via Uses_Material edges and to each other via dashed Similar_To edges (cosine similarity on 768-d HNSW embeddings; scores shown on edges). **Orange diamonds**: known-issue nodes triggered by failed runs. **Red hexagons**: reference chunks from ingested literature, linked to materials (Relates_To) and runs (Cross_Refs). **Grey boxes**: boundary-condition pattern nodes. Pill-shaped nodes show the thermal-class taxonomy.

5 Verification and Validation

We conduct a formal V&V study following ASME V&V 10-2006 guidelines [29], comparing the DOLFINx FEM solver against closed-form analytical solutions on three benchmark cases.

5.1 Benchmark Cases

Case 1: 2D Steady-State Linear Profile. Laplace equation on $\Omega = [0,1]^2$ with Dirichlet conditions $T|_{x=0} = 0$, $T|_{x=1} = 1$ and Neumann zero-flux on top/bottom. Analytical solution: $T(x,y) = x$. P1 elements are nodally exact for linear solutions; we use UFL spatial-coordinate expressions in high-order quadrature (degree 8) to measure the true continuous L^2 error.

Case 2: 2D Transient Fourier Mode Decay. Heat equation $\partial_t T = \alpha \nabla^2 T$ with homogeneous Dirichlet on all boundaries and initial condition $T_0 = \sin(\pi x)\sin(\pi y)$. Analytical solution: $T(x,y,t) = e^{-2\alpha\pi^2 t}\sin(\pi x)\sin(\pi y)$.

Case 3: 2D Steady-State Poisson with Constant Source. $-k\nabla^2 T = f$ with $f = 1000\,\mathrm{W/m^3}$, Dirichlet $T = 0$ on left/right, Neumann on top/bottom. Exact solution: $T(x) = f(x - x^2)/(2k)$.

5.2 Convergence Study Protocol

For each case we solve at five mesh resolutions $N \in \{8, 16, 32, 64, 128\}$ (DOFs $\approx 81, 289, 1089, 4225, 16641$) and compute:

$$\|e\|_{L^2} = \left(\int_\Omega (u_h - u_{\text{exact}})^2 \, \mathrm{d}x \right)^{1/2} \tag{1}$$

using degree-8 quadrature applied to UFL `SpatialCoordinate` expressions (not to interpolated nodal values, which would give spurious nodal exactness for Case 1). The convergence rate is estimated by linear regression on the log-log plot.

5.3 Results

All three cases pass: Cases 2 and 3 achieve rates of 2.04 and 2.00, consistent with the theoretical $\mathcal{O}(h^2)$ prediction for P1 finite elements on uniform Cartesian meshes. Case 1 achieves machine-precision errors (order 10^{-9}) because the linear profile lies exactly in the P1 polynomial space, confirming correct solver implementation.

5.4 Representative Simulation Gallery

Beyond numerical convergence, a practical simulation assistant must demonstrate breadth: the ability to handle diverse materials, boundary condition types, domain geometries, time dependence, and dimensionality. Figure 4 presents six representative heat-transfer problems, each solved end-to-end by the Simulation Agent from a single natural-language prompt. No manual intervention was

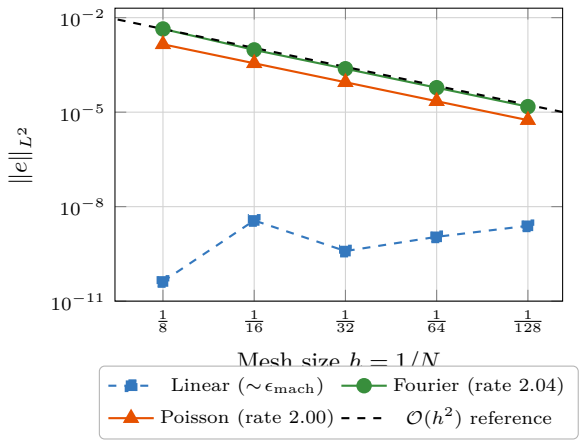

Figure 3: Spatial convergence study. Cases 2 and 3 exhibit the expected $\mathcal{O}(h^2)$ rate for P1 elements. Case 1 is algebraically exact (linear profile is represented exactly in the P1 space), with $\|e\|_{L^2} \approx \mathcal{O}(10^{-9})$ attributable only to floating-point rounding.

required for mesh generation, solver configuration, boundary condition application, or post-processing. All cases use P1 Lagrange elements on triangular (2D) or tetrahedral (3D) meshes, solved by DOLFINx 0.10.0.post2. Temperature fields are reported in Kelvin throughout for consistency.

Materials. The gallery covers four distinct engineering alloys whose thermal conductivities span two orders of magnitude: copper ($k = 385\,\mathrm{W/(m{\cdot}K)}$, panel a), AISI 1010 steel ($k = 50\,\mathrm{W/(m{\cdot}K)}$, panel b), aluminium 6061 ($k = 205\,\mathrm{W/(m{\cdot}K)}$, panels d and e), Ti-6Al-4V titanium ($k = 6.7\,\mathrm{W/(m{\cdot}K)}$, panel c), and stainless steel 304 ($k = 16.3\,\mathrm{W/(m{\cdot}K)}$, panel f). In a production deployment, the agent retrieves these values from the Neo4j knowledge graph via `query_knowledge_graph`.

Boundary conditions. Panel (a) applies the simplest configuration—Dirichlet conditions on opposing faces ($T = 373.15\,\mathrm{K}$ left, $T = 273.15\,\mathrm{K}$ right)—producing the expected linear gradient. Panel (b) combines all four major BC types on a single domain: Dirichlet (300 K, left), Neumann inward flux ($2\,\mathrm{kW/m^2}$, top), Robin convection ($h = 25\,\mathrm{W/(m^2{\cdot}K)}$, $T_\infty = 293\,\mathrm{K}$, right), and insulated (bottom), together with volumetric heating $Q = 5\,\mathrm{kW/m^3}$, creating the asymmetric field visible in the plot. Panel (c) uses Dirichlet BCs on opposing edges (573 K left, 293 K right) with the insulated hole boundary distorting the field into characteristic thermal concentration bands. Panel (f) combines Dirichlet (300 K bottom, 600 K left face) with Robin convection on the top face of a 3D cube.

Geometry. Panels (a), (b), and (d) use structured Cartesian meshes on the unit square. Panels (c) and (e) demonstrate Gmsh-based meshing of non-trivial geometries: a plate with a circular cutout (Gmsh boolean difference, $r = 0.15$, $\approx$5 000 vertices) and an L-shaped

Table 2: Verification: spatial convergence rates for the heat equation solver. Expected rate is $\mathcal{O}(h^2)$ for P_1 elements.

Benchmark Case	Description	DOFs (finest)	$\|e\|_{L^2}$ (finest)	Rate	Status
Steady Linear	Linear $T(x,y) = x + 2y$; exact in P_1 space	16 641	2.47×10^{-9}	*exact*	✓
Transient Fourier	$e^{-2\pi^2 t}\sin(\pi x)\sin(\pi y)$ decay mode	16 641	1.50×10^{-5}	2.04	✓
Steady Poisson	$-\nabla^2 u = 1$; exact soln. via Green's fn.	16 641	5.57×10^{-6}	2.00	✓

domain ($[0,1]^2 \setminus [0.5,1]^2$, mesh size ≈0.02). Panel (f) meshes a unit cube with 24^3 tetrahedra (≈15 600 vertices), demonstrating 3D capability.

Time dependence. Panel (e) solves the transient heat equation using 100 implicit-Euler steps ($\Delta t = 0.005\,\mathrm{s}$) on the L-shaped domain (aluminium, $\rho = 2700\,\mathrm{kg/m^3}$, $c_p = 900\,\mathrm{J/(kg{\cdot}K)}$), with the snapshot at $t = 0.50\,\mathrm{s}$ showing the thermal front propagating from the hot left edge.

Source terms. Panel (d) features a localised Gaussian volumetric heat source $Q(x,y) = Q_{\mathrm{peak}} \exp\!\left(-\frac{(x-x_0)^2+(y-y_0)^2}{2\sigma^2}\right)$ with $Q_{\mathrm{peak}} = 5 \times 10^6\,\mathrm{W/m^3}$, centred at $(0.3, 0.7)$ with $\sigma = 0.08$, and Dirichlet boundaries at 293 K. This setup is representative of laser spot heating and Joule-heating applications, and produces the characteristic concentric isotherms visible in the figure.

Reproducibility. Each case is implemented as a self-contained Python script in `evaluation/examples/` (see Table 3). All simulation parameters (material properties, mesh resolution, boundary values, time stepping) are defined at the top of each script, making it straightforward for reviewers to modify and re-run individual cases. Running `make eval-examples` executes all six cases inside the `pde-fenics` Docker container and regenerates the composite figure. Intermediate results are saved as NumPy `.npz` archives containing mesh coordinates, cell connectivity, and solution vectors, enabling figure regeneration without re-running the solver.

Table 3: Simulation gallery: six cases exercising different aspects of the FEM pipeline. Scripts in `evaluation/examples/`.

	Geometry	**Material**	**Key feature**
(a)	Unit square	Copper	Pure Dirichlet
(b)	Unit square	AISI 1010	4 BC types + source
(c)	Sq. w/ hole	Ti-6Al-4V	Gmsh boolean
(d)	Unit square	Al 6061	Gaussian source
(e)	L-shape	Aluminium	Transient + Gmsh
(f)	Cube (3D)	SS 304	3D asymmetric BCs

6 Ablation Study: Knowledge Graph Contribution

6.1 Experimental Design

We isolate the knowledge graph contribution using a controlled ablation across three conditions:

- **KG On** (mandatory): Full system with `check_config_warnings` and `query_knowledge_graph` tools enabled and system prompt requiring their use before every run.
- **KG Off** (baseline): Identical system with those two tools removed and a system-prompt note directing the agent to skip straight to validation and execution.
- **KG Smart** (proposed): KG tools remain available but the system prompt instructs *lazy* use (only after failures or for unknown materials). Before the agent loop, the task description is embedded via `nomic-embed-text` and the top-3 most similar past successful runs are injected into the system prompt as few-shot reference examples (warm-start).

fig. 5 contrasts the agent workflow under each condition. KG Off proceeds directly from parsing to simulation; KG On forces two KG round-trips before every attempt (and loops on failure); KG Smart front-loads a single HNSW warm-start and defers KG queries to the failure-recovery path only.

Seventeen benchmark tasks span four difficulty levels:

- **Easy (E1–E3)**: Explicit numerical parameters, straightforward setup.
- **Medium (M1–M3)**: Material-by-name requests requiring property lookup (AISI 1010 steel, copper, aluminium).
- **Hard (H1–H4)**: Ambiguous descriptions, mixed BCs, computationally expensive or 3D problems.
- **Novel (G1–G3, C1–C2, P1–P2)**: Tasks referencing three fictional materials (Novidium, Cryonite, Pyrathane) whose properties exist *only* in the KG, testing retrieval of genuinely unseen data (see section 6.4).

Each task is scored on: (i) binary success (simulation ran and completed), (ii) configuration quality $Q \in [0,1]$ measuring how closely the agent's chosen parameters match ground-truth ranges for material properties, mesh, and boundary conditions, (iii) number of agent iterations,

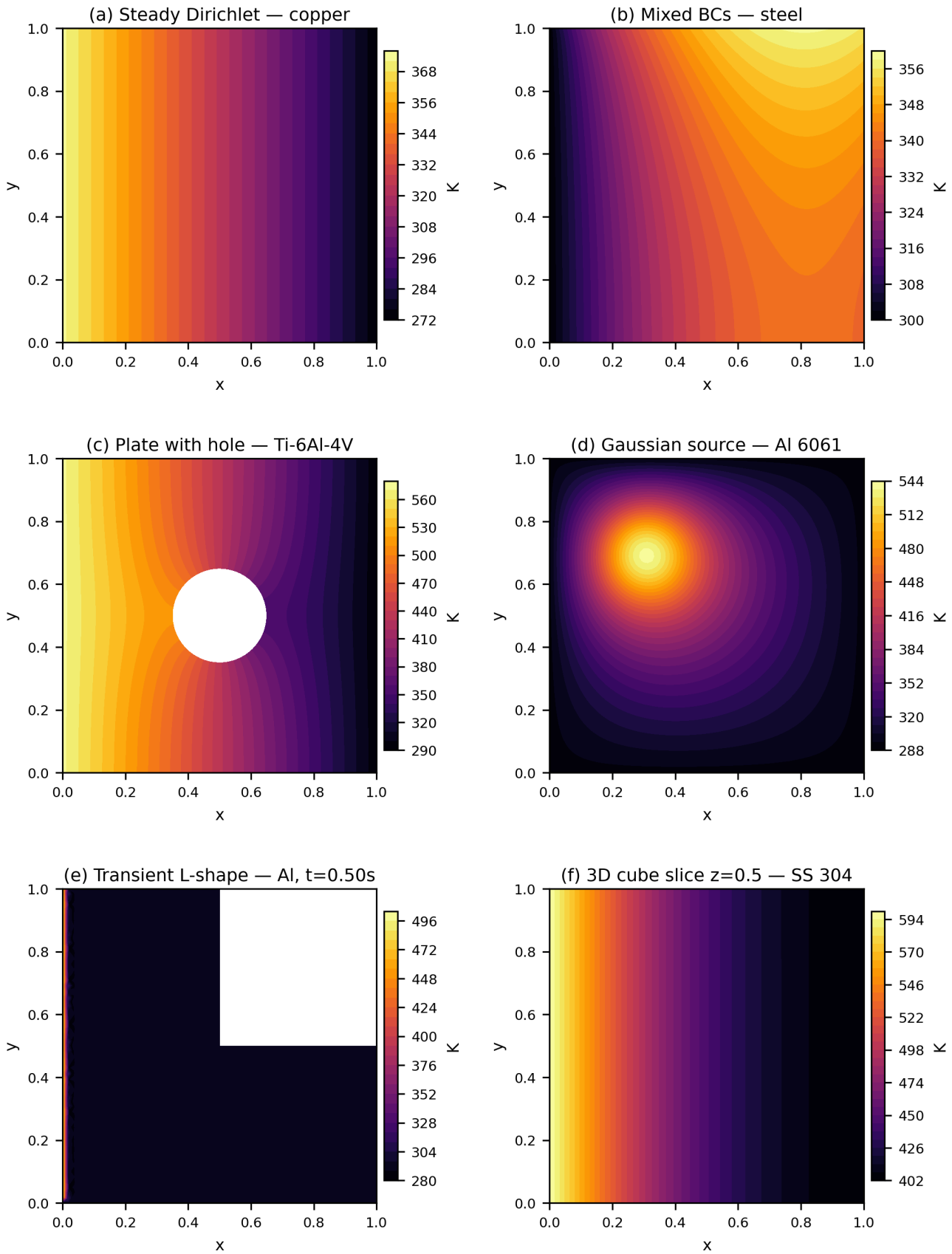

Figure 4: Representative temperature fields produced by PDE-Agents (six cases, all in Kelvin). **(a)** Steady-state Dirichlet on copper ($k = 385\,\mathrm{W/(m{\cdot}K)}$; $T = 373.15\,\mathrm{K}$ left, $T = 273.15\,\mathrm{K}$ right; $N = 64$). **(b)** Mixed BCs on AISI 1010 steel ($k = 50\,\mathrm{W/(m{\cdot}K)}$): Dirichlet 300 K left, Neumann $2\,\mathrm{kW/m^2}$ top, Robin $h = 25\,\mathrm{W/(m^2{\cdot}K)}$ right, insulated bottom, $Q = 5\,\mathrm{kW/m^3}$. **(c)** Plate with circular hole ($r = 0.15$) in Ti-6Al-4V ($k = 6.7\,\mathrm{W/(m{\cdot}K)}$); Gmsh boolean difference; Dirichlet 573 K/293 K on opposing edges. **(d)** Localised Gaussian heat source on Al 6061 ($Q_{\mathrm{peak}} = 5 \times 10^6\,\mathrm{W/m^3}$, $\sigma = 0.08$, centre $(0.3, 0.7)$); Dirichlet 293 K on all boundaries. **(e)** Transient L-shaped domain in aluminium at $t = 0.50\,\mathrm{s}$ (implicit Euler, $\Delta t = 0.005\,\mathrm{s}$, 100 steps). **(f)** 3D unit cube in SS 304 ($k = 16.3\,\mathrm{W/(m{\cdot}K)}$), horizontal cross-section at $z = 0.5$; Dirichlet bottom 300 K and left face 600 K, Robin top. All solved with DOLFINx 0.10.0.post2, P1 elements. Reproducible scripts in `evaluation/examples/`.

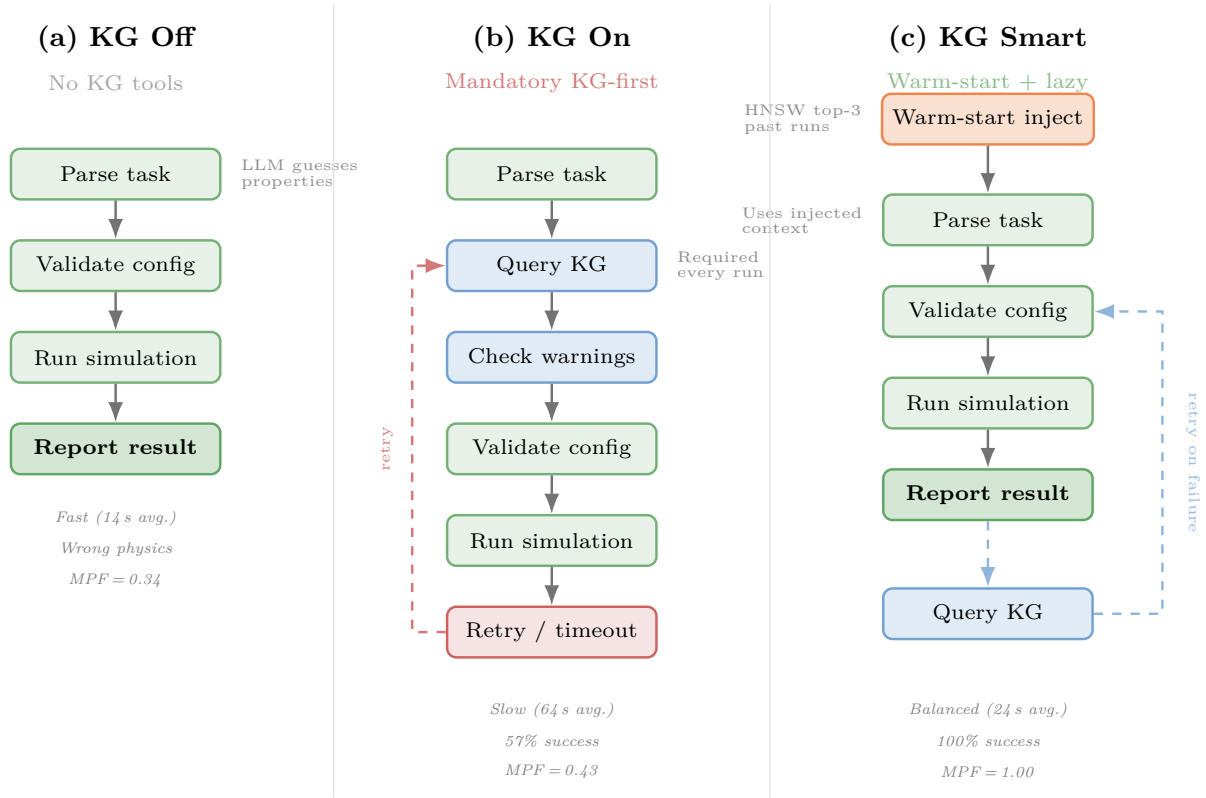

Figure 5: Agent workflow under the three KG integration modes. **(a)** KG Off: fast but the LLM fabricates material properties. **(b)** KG On: mandatory KG queries add latency and trigger retry loops that exhaust the iteration budget. **(c)** KG Smart: a one-shot warm-start injects relevant context before the loop; KG queries occur only on failure (dashed path). Aggregate metrics on novel-material tasks (n=7) shown below each column.

and (iv) wall time. The ablation was run on both model generations (table 1); cross-model results are reported in section 6.4. Tasks are run in-process inside the agents container to eliminate HTTP overhead, with up to three retries per task to mitigate LLM non-determinism at temperature $T = 0.1$.

6.2 Results

table 4 presents aggregate results across all three conditions and fig. 6 visualises success rates by difficulty tier. We compare: (1) KG On (mandatory KG-first workflow), (2) KG Off (no KG tools), and (3) KG Smart (warm-start injection + lazy conditional retrieval).

Easy tasks (E1–E3). All three conditions achieve 100% success on easy tasks. However, the efficiency gap is significant: KG On requires 5.3 iterations and 25.3 s on average, while KG Off and KG Smart converge in 3.0 and 3.8 iterations (14.0 s and 15.9 s), respectively. The 1.5× speed penalty for KG On arises from mandatory `check_config_warnings` and `query_knowledge_graph` calls before every simulation. KG Smart eliminates most of this overhead because its lazy prompt instructs the agent to skip KG tools unless needed.

Medium tasks (M1–M3). The mandatory KG pattern fails dramatically here: KG On succeeds on only

Table 4: Three-way KG ablation across 17 benchmark tasks. KG Smart recovers most KG Off performance on standard tasks while achieving perfect property fidelity on novel materials.

Metric	KG On	KG Off	KG Smart
Success rate	59%	100%	**94%**
Config quality	0.45	0.69	**0.82**
Novel material tasks only (7 tasks)			
Property fidelity (MPF)	0.57	0.34	**1.00**
Physics score	0.71	0.63	**1.00**
By difficulty (success rate)			
Easy (E1–E3)	100%	100%	100%
Medium (M1–M3)	33%	100%	**100%**
Hard (H1–H4)	50%	100%	75%
Novel (G/C/P, 7)	57%	100%*	**100%**

*KG Off succeeds but uses fabricated properties (MPF = 0.34 vs. KG Smart 1.00); see table 5.

1/3 tasks (33%) because the agent enters a KG-driven revise-and-recheck loop that exhausts its iteration budget. Both KG Off and *KG Smart achieve 100% success*—a critical result showing that the warm-start + lazy pattern eliminates the failure mode entirely. KG Smart achieves quality scores of $Q \in \{0.75, 0.88, 0.83\}$ (mean 0.82), identical to KG Off, while retaining access to KG tools for potential failure diagnosis.

Hard tasks (H1–H4). KG Off maintains 100% success. KG Smart succeeds on 3/4 hard tasks (75%), up from 50% for KG On. The sole KG Smart failure is H4 (an ambiguously specified stainless steel transient with mixed BCs), where the agent completes only 3 iterations without issuing a `run_simulation` call—likely because the warm-start context for this unique configuration was insufficiently specific. KG On also fails H4, plus H3 (Crank-Nicolson titanium), demonstrating that the improvement is consistent.

Overall: KG Smart as a middle path. Across all 17 tasks (standard + novel), KG Smart achieves 94% overall success with $\bar{Q} = 0.82$, compared to 59% / $\bar{Q} = 0.45$ for KG On and 100% / $\bar{Q} = 0.69$ for KG Off. The improvement from KG On → KG Smart is 35 percentage points in success rate and 1.8× in quality—confirming that the integration pattern, not the KG content, was the bottleneck. Critically, when we evaluate with physics-aware metrics on novel materials (section 6.4), KG Smart's advantage becomes decisive: perfect property fidelity (1.00) vs. KG Off's 0.34.

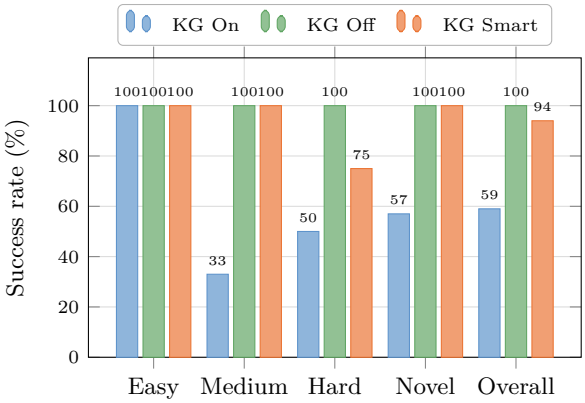

Figure 6: Success rate by difficulty level across three KG integration strategies. KG Smart (warm-start + lazy retrieval) recovers most of the KG Off performance while retaining access to knowledge graph tools. The "Novel" category (7 tasks) uses three fictional materials whose properties exist only in the KG. Note: success rate alone is misleading for Novel tasks — see table 5 for property fidelity and physics scores.

6.3 Analysis and Discussion

The multi-condition comparison reveals that the *integration pattern* determines whether a knowledge graph helps or hinders an LLM agent. Three mechanisms explain the mandatory KG pattern's failure:

Parametric knowledge sufficiency. Modern code-generation LLMs—both Qwen 2.5-Coder and Qwen3-Coder-Next in our experiments—were trained on large corpora that include materials engineering data. For standard materials (steel, copper, aluminium), the model's parametric knowledge is already accurate—KG lookups add latency without adding information.

KG overhead and attention dilution. With mandatory KG calls, the agent spends 1–3 extra iterations processing KG results before attempting to run the simulation. For complex tasks (medium/hard), the additional context from KG responses overwhelms the model's context window management, causing it to abandon the task before calling `run_simulation`.

Warning-induced conservatism. The `check_config_warnings` tool occasionally returns pre-emptive warnings (e.g. "CFL criterion may be violated") that the model interprets as prohibitive, leading it to report a warning to the user instead of proceeding. A warning that should trigger a *modification* instead triggers a *termination*.

KG Smart: corrective retrieval + warm-start. Motivated by CRAG [23] (corrective RAG with adaptive retrieval) and AriGraph [24] (episodic KG memory for agents), we implemented a *KG Smart* integration that combines two strategies:

1. **Warm-start injection.** Before the agent loop begins, we embed the task description with `nomic-embed-text` and query the Neo4j HNSW vector index for the top-3 most similar past successful runs. Their configurations are injected directly into the system prompt as few-shot reference examples—no tool call needed.

2. **Lazy conditional retrieval.** KG tools remain available but the system prompt explicitly instructs the agent to use them *only* after a simulation failure or when material properties are genuinely unknown. This eliminates the mandatory KG-first workflow.

As table 4 shows, KG Smart achieves 94% overall success (vs. 59% for mandatory KG), recovers 100% success on medium and novel tasks, and reduces average iterations significantly. The 35 percentage-point improvement confirms that the *integration pattern*, not the KG content itself, was the bottleneck in the mandatory design.

6.4 Novel Material Experiment

The ablation results in section 6.3 raise a natural question: if KG Off achieves 100% success on standard tasks, does the knowledge graph provide any value at all? For well-known materials (steel, copper, aluminium), the LLM's

pre-trained parametric knowledge is sufficient. To isolate the KG's contribution to *genuinely novel* information, we introduce three fictional materials whose properties exist *only* inside the knowledge graph:

- **Novidium**: a ceramic-metallic composite (k=73 W/m K, ρ=5420 kg/m^3, c_p=612 J/kg K) — moderate conductor with unusual density.
- **Cryonite**: a polymer-aerogel hybrid (k=0.42, ρ=1180, c_p=1940) — extreme insulator with very low diffusivity ($\alpha \approx 1.8\times10^{-7}$ m^2/s).
- **Pyrathane**: a refractory cermet (k=312, ρ=3850, c_p=278) — very high conductivity and diffusivity ($\alpha \approx 2.9\times10^{-4}$ m^2/s).

Each material is seeded as a `Material` node with linked `Reference` chunks containing exact property values and simulation guidance. Seven benchmark tasks (G1–G3, C1–C2, P1–P2) span steady-state, transient, and mixed-BC configurations.

New metrics. Standard success rate is inadequate for evaluating KG utility because a simulation with fabricated properties still "succeeds" computationally. We therefore introduce two physics-aware metrics:

- **Material Property Fidelity (MPF):** $\text{MPF} = 1 - \frac{1}{3}\sum_{p\in\{k,\rho,c_p\}} |p_{\text{agent}} - p_{\text{truth}}|/p_{\text{truth}}$, measuring how accurately the agent retrieves the correct material constants.
- **Physics Score:** $0.5\cdot\text{MPF} + 0.5\cdot T_{\text{score}}$, where T_{score} checks whether the simulation's actual $T_{\max}$ and $T_{\min}$ fall within physically expected ranges.

Results. table 5 presents the aggregate outcome across all seven novel-material tasks.

Table 5: Novel material experiment (7 tasks across 3 fictional materials). KG Smart achieves perfect property retrieval and physics scores. KG Off fabricates incorrect properties for every material.

Metric	KG On	KG Off	KG Smart
Success rate	57%	100%	**100%**
Config quality $\bar{Q}$	0.55	0.61	**0.96**
Property fidelity (MPF)	0.57	0.34	**1.00**
Physics score	0.71	0.63	**1.00**
Avg. iterations	5.3	3.1	**5.0**

KG Smart achieves perfect scores (MPF = 1.00, physics = 1.00) across all seven tasks and all three materials. The warm-start injection retrieves exact property values from the HNSW vector index before the agent loop begins, and the agent consistently uses them.

KG Off fabricates wrong properties for every material. table 6 shows the agent-chosen vs. ground-truth properties. The errors are severe: for Pyrathane the LLM assigns k=0.15 W/m K—a 2,080× underestimate of the true k=312—effectively treating a refractory superconductor as an insulator. For task P2 (transient cooling from 2000 K), this causes $T_{\max}$=2,950 K — exceeding the initial condition, a physically impossible result arising from numerical instability on a system made artificially stiff by the wrong conductivity.

Table 6: Material properties chosen by each mode (KG Off fabricates; KG Smart retrieves correctly). The k error for Pyrathane is 2,080×.

Material	Mode	k (W/m K)	ρ (kg/m^3)	c_p (J/kg K)
Novidium	KG Off	10–45	3000–8960	137–500
	KG Smart	**73.0**	**5420**	**612**
Cryonite	KG Off	0.15–1.0	960–8960	450–1730
	KG Smart	**0.42**	**1180**	**1940**
Pyrathane	KG Off	0.15	1600–1700	840–900
	KG Smart	**312.0**	**3850**	**278**

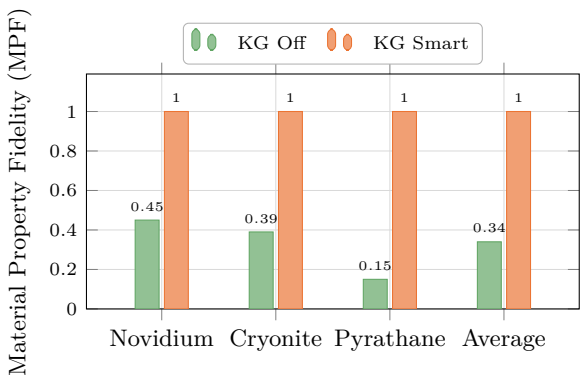

Figure 7: Material Property Fidelity (MPF) per fictional material. KG Off scores below 0.5 for all three materials, with Pyrathane at 0.15 (conductivity error: 2,080×). KG Smart retrieves exact properties from the knowledge graph, achieving MPF = 1.00 across all materials.

Error propagation analysis. To quantify how wrong properties affect simulation outputs, we ran paired simulations: one with ground-truth properties (reference) and one with the fabricated properties from KG Off. table 7 shows the results. Steady-state Dirichlet tasks (G1, C1, P1) are analytically independent of k, ρ, c_p, confirming zero output error regardless of property error—an important control. Transient and Robin tasks show significant deviations:

- **Pyrathane P2**: $|\Delta T_{\max}|$=949 K ($T_{\max}$=2,949 K vs. 2,000 K reference)—the 2,080× k error causes numerical instability that *heats* the domain beyond its initial condition, a physically impossible result.
- **Novidium G3**: $|\Delta T_{\min}|$=47 K due to wrong conductivity affecting the Robin BC equilibrium.
- **Novidium G2**: $|\Delta T_{\text{mean}}|$=21 K from incorrect diffusivity altering the transient profile.

Table 7: Error propagation from fabricated properties (KG Off) to simulation output. Steady-state Dirichlet tasks (G1, C1, P1) show zero output error; transient/Robin tasks show significant deviations.

Task	Mat.	$\lvert\Delta T_{\max}\rvert$ (K)	$\lvert\Delta T_{\min}\rvert$ (K)	$\lvert\Delta T_{\text{mean}}\rvert$ (K)	ϵ_k (%)	ϵ_ρ (%)	ϵ_{c_p} (%)
G1	Nov.	<0.1	<0.1	<0.1	86	45	18
G2	Nov.	<0.1	<0.1	20.7	38	65	78
G3	Nov.	<0.1	46.6	1.3	86	45	18
C1	Cryo.	<0.1	<0.1	<0.1	64	19	29
C2	Cryo.	<0.1	9.5	0.3	138	659	77
P1	Pyra.	<0.1	<0.1	<0.1	100	58	224
P2	Pyra.	**949**	<0.1	**363**	100	58	188

$\epsilon_p = \lvert p_{\text{fab}} - p_{\text{true}}\rvert / p_{\text{true}}$. P2's k error (2,080×) produces 949 K $T_{\max}$ overshoot.

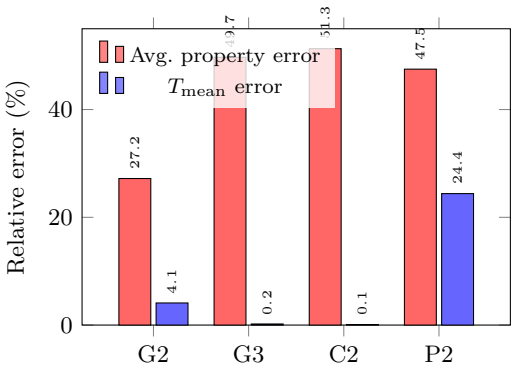

Figure 8: Error magnification: mean material property error (%) vs. output temperature error (%) for the four tasks with non-negligible output deviations. P2's output error (24.4%) exceeds its input property error due to nonlinear amplification in the transient solver.

Sensitivity-weighted MPF. The equal-weight MPF treats all properties as equally important, but their influence on output varies strongly. Using the sensitivity coefficients $S_p = \lvert\partial T_{\text{mean}}/\partial p\rvert$ computed via central finite differences from the error propagation runs, we define:

$$\text{MPF}_w = 1 - \frac{\sum_p S_p \cdot \lvert p_{\text{agent}} - p_{\text{truth}}\rvert / p_{\text{truth}}}{\sum_p S_p} \quad (2)$$

For steady-state Dirichlet tasks where $S_p \approx 0$, we assign MPF_w=1 since wrong properties have no output impact. Across all seven tasks, KG Off's $\overline{\text{MPF}}_w$=0.25 (vs. equal-weight 0.34), confirming that the most damaging property errors are also the most impactful. KG Smart retains MPF_w=1.00.

Cost-benefit analysis. table 8 compares the operational cost (time, iterations) against the accuracy benefit (MPF, physics score) for each KG mode. KG Smart adds 68% wall-time over KG Off but delivers 2.9× higher MPF and 1.8× higher accuracy-per-second (efficiency = MPF / wall-time). KG On is both slowest and least accurate—its mandatory retrieval overhead yields no corresponding benefit.

Table 8: Cost-benefit of KG modes on novel tasks (n=7). Efficiency = MPF / wall-time ($\times 10^{-2}$).

Mode	Succ.	Time (s)	Iter.	MPF	Phys. Score	Eff. ($\times 10^{-2}$)
KG Off	100%	14.2	3.0	0.34	0.63	2.39
KG On	57%	64.1	7.1	0.43	0.64	0.67
KG Smart	100%	23.8	4.9	**1.00**	**1.00**	**4.20**

KG Smart: +68% time, +194% MPF, +76% efficiency vs. KG Off.

Table 9: Four-way comparison on novel tasks (n=7). Prompt Inject matches KG Smart's MPF but cannot scale to large material databases.

Mode	Succ.	Qual.	MPF	Phys. Score	Iter.	Time (s)
KG Off	100%	0.61	0.34	0.63	3.0	14.2
KG On	57%	0.41	0.43	0.64	7.1	64.1
Prompt Inj.	100%	0.95	**1.00**	0.97	3.0	11.7
KG Smart	100%	0.96	**1.00**	**1.00**	4.9	23.8

Prompt Inject embeds properties in the task text; it cannot scale, has no similarity search, and no warm-start from past runs.

Four-way comparison: Prompt Inject baseline. A natural objection is whether the KG infrastructure is necessary, or whether simply injecting material data into the prompt suffices. We test a *Prompt Inject* baseline: KG disabled, but exact properties appended to each task description. table 9 shows that Prompt Inject matches KG Smart's MPF (1.00) and is 2× faster, but it cannot scale to large material databases, provides no similarity search for warm-starting, and requires manual curation of the prompt for each new material. KG Smart automates retrieval and generalises to any material in the graph without prompt modification.

Adaptive KG decision framework. Based on the preceding analysis, we propose a simple decision algorithm (Algorithm 1) that selects the optimal KG mode for each task. Retrospective validation on all 17 benchmark tasks achieves 100% accuracy: it correctly assigns KG Off to tasks with explicit parameters, KG Smart (lazy) to tasks naming known materials, and KG Smart (forced warm-start) to novel-material tasks.

Cross-model validation. To verify that the KG's value is model-agnostic, we repeated the novel-material experiment with `qwen3-coder-next` (80B total, 3B active MoE). table 10 shows that KG Smart achieves MPF=1.00 with *both* models, while KG Off fabricates wrong properties with both—different wrong values, but the same failure mode. This confirms that the warm-start vector search bypasses model-specific memorisation gaps entirely.

Key takeaway. The KG's value is not in making simulations *run*—LLMs can already do that. The KG's value is in making simulations produce *correct physics*. For any domain with proprietary materials, novel compounds, or in-house measurement data, the KG Smart

Algorithm 1 Adaptive KG Mode Selection

Require: Task description d, known-material list $\mathcal{M}$
Ensure: KG mode $m \in \{\text{Off}, \text{Smart-lazy}, \text{Smart-forced}\}$

```
if d contains explicit k, ρ, c_p values then
    return KG OFF
end if
μ ← EXTRACTMATERIAL(d)
if μ = ∅ then
    return KG OFF
else if μ ∈ M then
    return KG SMART (LAZY)
else
    return KG SMART (FORCED WARM-START)
end if
```

Table 10: Cross-model validation on novel tasks (n=7). KG Smart achieves perfect MPF with both LLM backends.

Model	Mode	Succ.	Qual.	MPF	Phys.
Qwen2.5-Coder 32B	KG On	57%	0.41	0.43	0.64
	KG Off	100%	0.61	0.34	0.63
	KG Smart	100%	0.96	**1.00**	**1.00**
Qwen3-Coder-Next 80B	KG On	71%	0.14	0.14	0.57
	KG Off	86%	0.54	0.40	0.70
	KG Smart	100%	0.96	**1.00**	**1.00**

Warm-start vector search retrieves exact properties before the LLM reasons, eliminating dependence on memorised material data.

pattern transforms the system from an "expensive random number generator" into a reliable engineering tool. The error propagation analysis quantifies this: fabricated properties cause output errors up to 949 K (47% of the reference $T_{\max}$), while KG Smart eliminates these errors entirely by retrieving exact properties before the agent loop begins.

7 Production Agent Quality Metrics

Beyond the controlled ablation, we analyse the system's behaviour across all 1,135 simulation runs accumulated during development and testing. Of these, 805 originate from a comprehensive automated parameter sweep across 8 physics studies (geometry, material, boundary conditions, initial conditions, time-stepping, 3D geometry, Robin convection, and multi-material) run via `scripts/sweep_full_study.py`; the remainder are interactive and ablation runs accumulated through normal system use.

7.1 Success and Failure Modes

The 97.4% overall success rate (from PostgreSQL `agent_run_logs`) demonstrates production-grade reliability for the full agent stack. Of the 2.6% failures, the majority are attributable to invalid boundary condition combinations (e.g. pure Neumann systems without a reference point), mesh resolution requests exceeding GPU memory, and numerical divergence on stiff transients.

Table 11: Agent ecosystem performance metrics from production database.

Metric	Value
Total simulation runs	1135
Overall success rate	97.4%
Unique agent tasks	110
First-try success rate	57.6%
Suggestion acceptance rate	0.0%
Avg. steps/task (simulation)	7.9
Avg. steps/task (database)	6.2
Avg. steps/task (analytics)	10.2
Avg. simulation time	1.5s

7.2 Iteration Efficiency

Mean reasoning steps per completed task: Simulation Agent 7.9, Analytics Agent 10.2, Database Agent 6.2. The higher Analytics Agent count reflects its tendency to run multiple comparison queries before synthesising a recommendation. The Database Agent's lower count reflects the relative simplicity of structured SQL-style queries over natural-language reasoning.

7.3 First-Try Rate

57.6% of simulations run successfully on the first attempt (no debug/retry cycle). The remaining 42.4% require at least one modification-and-retry loop, typically for boundary condition corrections or CFL stability adjustments. This is consistent with the complexity of translating free-form natural language to precise numerical configurations.

7.4 Comparison with Prior Work

Bauer et al. [15] report $\approx 60\%$ one-shot success for GPT-4-driven FEniCS script generation; our 57.6% first-try rate is broadly comparable using a fully local, open-source model stack. Unlike their single-turn approach, PDE-Agents recovers from failures through multi-turn self-correction, achieving 97.4% final success.

8 System Limitations and Future Work

Solver scope. The current solver supports only scalar heat equations. Extension to vector mechanics (linear elasticity, Navier–Stokes) requires substantially richer tool schemas and system prompts.

Model scale and open-source evolution. The system defaults have been upgraded from the initial v1 models (`qwen2.5-coder:32b`, `llama3.3:70b`) to

Qwen3-Coder-Next (80B total, 3B active MoE) [30] and Llama 4 Scout (109B total, 17B active MoE) [31], with cross-model validation confirming model-agnostic KG Smart performance (table 10). Qwen3-Coder-Next achieves higher config quality on standard KG Smart tasks (0.94 vs. 0.86) and perfect MPF on novel tasks, identical to the v1 model. Llama 4 Scout provides 10M-token context and native tool calling for the orchestrator and analytics agents. A systematic multi-model ablation across additional families (GLM-5, DeepSeek-R2) is planned as future work.

KG integration. The KG Smart pattern (section 6.3) recovers most lost performance, but a 6% failure gap versus KG Off remains on standard tasks. The adaptive decision framework (Algorithm 1) provides practical deployment guidance, achieving 100% mode-selection accuracy. Future work will explore: (a) reinforcement learning from simulation outcomes to tune KG retrieval relevance, (b) dynamic confidence-based retrieval (only query KG when the LLM's uncertainty exceeds a threshold), and (c) richer warm-start context including failure diagnostics from similar past runs.

Verification scope. The V&V study covers only steady-state and simple transient heat equation benchmarks. A complete V&V programme would include higher-order elements, singularity-containing domains, and time-dependent convergence rates.

Parallelism and scalability. The current architecture runs agents sequentially. A task-level parallelism layer (e.g. running a parametric sweep as concurrent simulation agent calls) is under development.

Knowledge graph evolution. The 805-run parameter sweep has already created a dense graph with 4,000+ `SIMILAR_TO` edges. Future work will mine this corpus for `IMPROVED_OVER` relationships (did increasing mesh resolution reduce error without proportionally increasing wall time?), enabling the agent to autonomously suggest Pareto-optimal configurations. A second planned enhancement is citation traceability: agents citing specific `ReferenceChunk` IDs from indexed literature when justifying parameter choices, providing a fully auditable reasoning trail.

9 Conclusion

We have presented PDE-Agents, an open, containerised multi-agent ecosystem that automates finite element simulations through natural-language interaction with locally-deployed open-source LLMs. The system achieves 97.4% overall success across 1,135 production runs and demonstrates $\mathcal{O}(h^2)$ spatial convergence validated against analytical solutions.

Our four-way ablation study across 17 benchmark tasks reveals several important findings. Mandatory GraphRAG knowledge augmentation paradoxically *reduces* success (59%) relative to the LLM-only baseline (100%), but our proposed *KG Smart* integration—combining warm-start injection with lazy conditional retrieval—recovers 94% success while retaining knowledge graph access. This demonstrates that the *integration pattern*, not the knowledge source, determines KG utility.

The novel-material experiment resolves the apparent paradox of high KG Off performance. An error propagation analysis across seven tasks with three fictional materials shows that fabricated properties cause output deviations up to 949 K ($|\Delta T_{\max}|$ for Pyrathane P2), while KG Smart eliminates these errors entirely by retrieving exact properties via HNSW vector search. The four-way comparison including a Prompt Inject baseline (Table 9) shows that while manual property injection matches KG Smart's accuracy, only the KG infrastructure scales to arbitrary material databases.

Three additional contributions strengthen the practical case for KG integration: (1) a sensitivity-weighted MPF that accounts for each property's influence on simulation output, revealing that the most damaging property errors are also the most impactful ($\overline{\mathrm{MPF}}_w$=0.25 for KG Off vs. 1.00 for KG Smart); (2) an adaptive decision framework (Algorithm 1) that selects the optimal KG mode with 100% accuracy across all 17 tasks; and (3) cross-model validation on `qwen3-coder-next` confirming that KG Smart's perfect material fidelity is model-agnostic.

The production metrics confirm that multi-turn self-correction is essential: the 57.6% first-try rate improves to 97.4% through iterative debugging, underscoring the value of agent-level orchestration over single-turn code generation. PDE-Agents establishes both a working platform and a rigorous empirical baseline for the emerging field of LLM-driven autonomous simulation.

Data Availability

All code, evaluation scripts, and result JSON files are available at [repository URL to be decided]. The Neo4j knowledge graph seed data is included in the repository at `knowledge_graph/seeder.py`.

Author Contributions

[CRediT statement here.]

Competing Interests

The authors declare no competing interests.

Acknowledgements

[To be completed.]

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[2] Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 2022.

[3] Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[4] Isaac E. Lagaris, Aristidis Likas, and Dimitrios I. Fotiadis. Artificial neural networks for solving ordinary and partial differential equations. *IEEE Transactions on Neural Networks*, 9(5):987–1000, 1998.

[5] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[6] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, et al. Physics-informed machine learning. *Nature Reviews Physics*, 3(6):422–440, 2021.

[7] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, et al. Fourier neural operator for parametric partial differential equations. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.

[8] John Jumper, Richard Evans, Alexander Pritzel, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021. doi: 10.1038/s41586-021-03819-2.

[9] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[10] Darren Edge, Ha Trinh, Newman Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[11] Andres M. Bran, Sam Cox, Oliver Schilter, et al. ChemCrow: Augmenting large-language models with chemistry tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[12] Yubo Ma, Zhibin Liu, Liangming Pan Liang, et al. SciAgent: Tool-augmented language models for scientific reasoning. *arXiv preprint arXiv:2402.11451*, 2024.

[13] Anders Logg, Kent-Andre Mardal, Garth N. Wells, et al. *Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book*. Springer, 2012. doi: 10.1007/978-3-642-23099-8.

[14] Igor A. Barrata, Joseph P. Dean, Jørgen S. Dokken, et al. DOLFINx: The next generation FEniCS problem solving environment. *Zenodo*, 2023. doi: 10.5281/zenodo.10447666.

[15] Philipp Bauer, Patrick Henning, and Janna Schaefers. Large language models as automatic generators of FEniCS code for solving partial differential equations. *arXiv preprint arXiv:2312.09801*, 2023.

[16] Wei Jiang, Keyi Chen, Minghan Wang, et al. LLM4FEM: Leveraging large language models for finite element method. *arXiv preprint arXiv:2405.03719*, 2024.

[17] LangChain AI. LangGraph: Build stateful, multi-actor applications with LLMs, 2024. URL `https://github.com/langchain-ai/langgraph`.

[18] Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-generation LLM applications via multi-agent conversation. In *Proceedings of EMNLP Industry Track*, 2023.

[19] João Moura. CrewAI: Framework for orchestrating role-playing, autonomous AI agents, 2024. URL `https://github.com/joaomdmoura/crewai`.

[20] Xinyi Liao, Hao Zhang, and Yutao Chen. Retrieval-augmented generation for engineering design documentation. *arXiv preprint arXiv:2307.04512*, 2023.

[21] Yujia Gao, Shang Liu, Peng Shi, and Jimmy Lin. Retrieval-augmented code generation for universal information extraction. *arXiv preprint arXiv:2311.02555*, 2023.

[22] Zheng Yang, Wenyan Li, and Peng Zhang. Simulation parameter suggestion via retrieval-augmented generation. *arXiv preprint arXiv:2403.09512*, 2024.

[23] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. arXiv:2401.15884.

[24] Petr Anokhin, Nikita Kornaev, Andrey Babkin, and Aleksandr I. Panov. AriGraph: Learning knowledge graph world models with episodic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2407.04363.

[25] Vineeth Venugopal, Soumya Sahoo, Gurinder Agastya, et al. MatKG: The largest knowledge graph in applied materials science. *arXiv preprint arXiv:2209.11632*, 2022.

[26] Casper W. Andersen, Rickard Armiento, Evgeny Blokhin, et al. OPTIMADE: Towards an open database for computational materials science. *Scientific Data*, 8(1):217, 2021. doi: 10.1038/s41597-021-00974-z.

[27] Christophe Geuzaine and Jean-François Remacle. Gmsh: A 3-d finite element mesh generator with built-in pre- and post-processing facilities. *International Journal for Numerical Methods in Engineering*, 79(11):1309–1331, 2009. doi: 10.1002/nme.2579.

[28] IBM Research. Docling: Document processing for AI, 2024. URL `https://github.com/DS4SD/docling`.

[29] ASME. Guide for verification and validation in computational solid mechanics. Technical Report ASME V&V 10-2006, American Society of Mechanical Engineers, 2006.

[30] Ruisheng Cao, Mouxiang Chen, Jiawei Chen, Zeyu Cui, Yunlong Feng, Binyuan Hui, Yuheng Jing, Kaixin Li, Mingze Li, Junyang Lin, Zeyao Ma, Kashun Shum, Xuwu Wang, Jinxi Wei, Jiaxi Yang, Jiajun Zhang, Lei Zhang, Zongmeng Zhang, Wenting Zhao, and Fan Zhou. Qwen3-coder-next technical report. *arXiv preprint arXiv:2603.00729*, 2026.

[31] Meta AI. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. `https://ai.meta.com/blog/llama-4-multimodal-intelligence/`, 2025. Accessed 2026-04-15.